# Sistemas distribuidos y Programación paralela

Medición de throughput.

Matt Zander
Gustavo Andrés Contardi
Valentín Joel Romero Monteagudo

## Comandos utilizados para las pruebas:

1. Buildear la imagen Docker a partir del Dockerfile y el código actual.

```
$ docker build -t servidor-hit2 .
```

2. Levantar el contenedor:

```
$ docker run -d \
  -p 5002:8080 \
  -e MAX_WORKERS=1 \
  -v /var/run/docker.sock:/var/run/docker.sock \
  --name servidor \
  servidor-hit2
```

3. Mandar un loop con un conjunto de tareas que lleguen al mismo tiempo:

```
$ for i in {1..10}; do
      curl -s -X POST http://localhost:5002/getRemoteTask \
      -H "Content-Type: application/json" \
      -d  '{"servicio":  "texto",  "payload":  {"texto":  "sistemas
      distribuidos"}, "lamport_ts": '$i'}' &
done
wait
```

4. Consultar las métricas:

```
$ curl http://localhost:5002/metricas
```

/metricas es un endpoint que agregamos nosotros en el servidor del #HIT 2, responde con un JSON que contiene tareas pendientes, tareas completadas, throughput por minuto y tiempo transcurrido.

## Ejemplo de prueba:

```
gustavo@gustavo: ~/Escritorio/Distribuidos/TP2_SDyPP/hit2/servidor
gustavo@gustavo: ~/Escritorio/Distribuidos/TP2_SDyPP/hit2/servi...   gustavo@gustavo: ~/Escritorio/Distribuidos/TP2_SDyPP/hit2/servi...
gustavo@gustavo:~/Escritorio/Distribuidos/TP2_SDyPP/hit2/servidor$ docker build -t servidor-hit2 .
[+] Building 1.0s (11/11) FINISHED                                                          docker:default
 => [internal] load build definition from Dockerfile                                                    0.0s
 => => transferring dockerfile: 762B                                                                    0.0s
 => [internal] load metadata for docker.io/library/python:3.12-slim                                     1.0s
 => [internal] load .dockerignore                                                                       0.0s
 => => transferring context: 2B                                                                         0.0s
 => [1/6] FROM docker.io/library/python:3.12-slim@sha256:3d5ed973e45820f5ba5e46bd065bd88b3a504ff072  0.0s
 => [internal] load build context                                                                       0.0s
 => => transferring context: 95B                                                                        0.0s
 => CACHED [2/6] WORKDIR /app                                                                           0.0s
 => CACHED [3/6] RUN apt-get update && apt-get install -y     curl     ca-certificates     --no-ins  0.0s
 => CACHED [4/6] COPY requirements.txt .                                                                0.0s
 => CACHED [5/6] RUN pip install --no-cache-dir -r requirements.txt                                     0.0s
 => CACHED [6/6] COPY app/servidor.py .                                                                 0.0s
 => exporting to image                                                                                  0.0s
 => => exporting layers                                                                                 0.0s
 => => writing image sha256:78b12562f4200feef847a901b39d583783c55f7fd6aea9805596b0d411b8adc4            0.0s
 => => naming to docker.io/library/servidor-hit2                                                        0.0s
gustavo@gustavo:~/Escritorio/Distribuidos/TP2_SDyPP/hit2/servidor$ docker run -d   -p 5002:8080   -e MAX_W
ORKERS=1   -v /var/run/docker.sock:/var/run/docker.sock   --name servidor   servidor-hit2
ebe31a50d74a10b7e111c8a6f8b85831328d1f8670d1e7ace97fedbb9865a8ec
gustavo@gustavo:~/Escritorio/Distribuidos/TP2_SDyPP/hit2/servidor$
```

```
gustavo@gustavo: ~/Escritorio/Distribuidos/TP2_SDyPP/hit2/servidor
gustavo@gustavo: ~/Escritorio/Distribuidos/TP2_SDyPP/hit2/servi...   gustavo@gustavo: ~/Escritorio/Distribuidos/TP2_SDyPP/hit2/servi...
gustavo@gustavo:~/Escritorio/Distribuidos/TP2_SDyPP/hit2/servidor$ for i in {1..10}; do   curl -s -X POST
http://localhost:5002/getRemoteTask     -H "Content-Type: application/json"     -d '{"servicio": "texto",
"payload": {"texto": "sistemas distribuidos"}, "lamport_ts": '$i'}' & done
[1] 52841
[2] 52842
[3] 52843
[4] 52844
[5] 52845
[6] 52846
[7] 52847
[8] 52848
[9] 52849
[10] 52850
gustavo@gustavo:~/Escritorio/Distribuidos/TP2_SDyPP/hit2/servidor$ {"lamport_ts":15,"resultado":{"resultad
o":"sodiubirtsid sametsis"},"servicio":"texto","tarea_id":"tarea-e2e03f"}
{"lamport_ts":16,"resultado":{"resultado":"sodiubirtsid sametsis"},"servicio":"texto","tarea_id":"tarea-43
5b01"}
{"lamport_ts":17,"resultado":{"resultado":"sodiubirtsid sametsis"},"servicio":"texto","tarea_id":"tarea-50
c638"}
{"lamport_ts":18,"resultado":{"resultado":"sodiubirtsid sametsis"},"servicio":"texto","tarea_id":"tarea-57
b9a5"}
{"lamport_ts":19,"resultado":{"resultado":"sodiubirtsid sametsis"},"servicio":"texto","tarea_id":"tarea-73
0dfe"}
{"lamport_ts":20,"resultado":{"resultado":"sodiubirtsid sametsis"},"servicio":"texto","tarea_id":"tarea-df
```

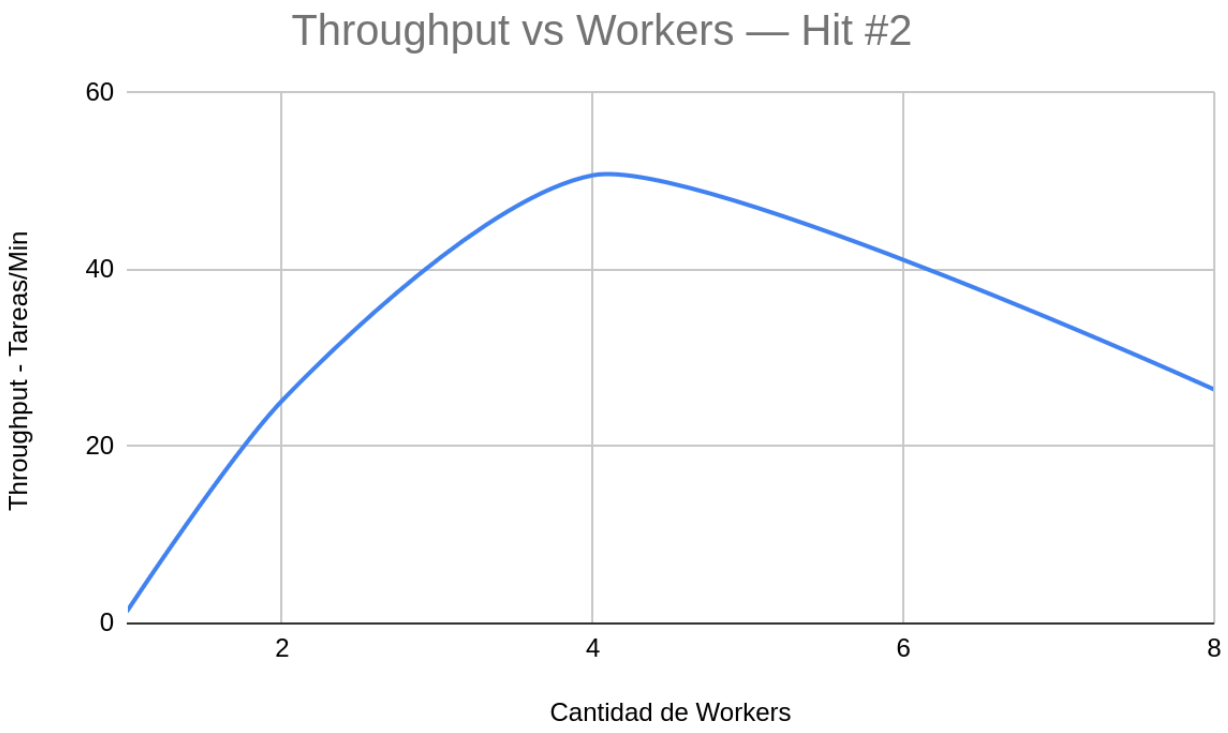
gustavo@gustavo: ~/Escritorio/Distribuidos/TP2_SDyPP/hit2/servidor

gustavo@gustavo: ~/Escritorio/Distribuidos/TP2_SDyPP/hit2/servi... | gustavo@gustavo: ~/Escritorio/Distribuidos/TP2_SDyPP/hit2/servi...

```
 => [internal] load build definition from Dockerfile                                                    0.0s
 => => transferring dockerfile: 762B                                                                    0.0s
 => [internal] load metadata for docker.io/library/python:3.12-slim                                     1.0s
 => [internal] load .dockerignore                                                                       0.0s
 => => transferring context: 2B                                                                         0.0s
 => [1/6] FROM docker.io/library/python:3.12-slim@sha256:3d5ed973e45820f5ba5e46bd065bd88b3a504ff072  0.0s
 => [internal] load build context                                                                       0.0s
 => => transferring context: 95B                                                                        0.0s
 => CACHED [2/6] WORKDIR /app                                                                           0.0s
 => CACHED [3/6] RUN apt-get update && apt-get install -y     curl     ca-certificates     --no-ins  0.0s
 => CACHED [4/6] COPY requirements.txt .                                                                0.0s
 => CACHED [5/6] RUN pip install --no-cache-dir -r requirements.txt                                     0.0s
 => CACHED [6/6] COPY app/servidor.py .                                                                 0.0s
 => exporting to image                                                                                  0.0s
 => => exporting layers                                                                                 0.0s
 => => writing image sha256:78b12562f4200feef847a901b39d583783c55f7fd6aea9805596b0d411b8adc4         0.0s
 => => naming to docker.io/library/servidor-hit2                                                        0.0s
gustavo@gustavo:~/Escritorio/Distribuidos/TP2_SDyPP/hit2/servidor$ docker run -d   -p 5002:8080   -e MAX_W
ORKERS=1   -v /var/run/docker.sock:/var/run/docker.sock   --name servidor   servidor-hit2
ebe31a50d74a10b7e111c8a6f8b85831328d1f8670d1e7ace97fedbb9865a8ec
gustavo@gustavo:~/Escritorio/Distribuidos/TP2_SDyPP/hit2/servidor$ curl http://localhost:5002/metricas
{"cola_pendiente":0,"tareas_completadas":10,"throughput_por_minuto":1.38,"tiempo_segundos":433.79,"workers
_max":1}
gustavo@gustavo:~/Escritorio/Distribuidos/TP2_SDyPP/hit2/servidor$
```

**Resultados:**

| Workers | Tareas completadas | Tiempo (s) | Throughput (tareas/min) |
|---|---|---|---|
| 1 | 10 | 433.79 | 1.38 |
| 2 | 10 | 23.79 | 25.22 |
| **4** | **10** | **11.84** | **50.69** |
| 8 | 10 | 22.68 | 26.45 |




El speedup no es lineal. De 1 a 4 workers el throughput crece fuertemente, casi duplicándose al duplicar la cantidad de workers. Sin embargo, al pasar a 8 workers el throughput cae aproximadamente a la mitad respecto al pico en 4 workers. Esto indica la presencia de un cuello de botella que aparece entre los 4 y 8 workers.

## Recursos compartidos y cuellos de botella

Al correr todo en un solo equipo, los recursos compartidos que pueden convertirse en cuello de botella son:

- **Docker daemon:** Proceso único que atiende todas las solicitudes de docker run y docker rm. Con 8 workers compitiendo simultáneamente para levantar contenedores, el daemon serializa las solicitudes internamente y se convierte en el limitante principal.
- **CPU:** Si las tareas son compute-intensive, los cores se saturan al aumentar la concurrencia. En este caso las tareas son livianas, pero el overhead del daemon sí consume CPU.
- **Memoria RAM:** Cada contenedor Docker consume memoria. Con muchos workers activos simultáneamente, la RAM disponible puede agotarse y el sistema comienza a usar swap, degradando el rendimiento.
- **I/O de disco:** Docker accede al disco al levantar cada contenedor. Múltiples arranques simultáneos generan contención en el subsistema de I/O.
- **Red interna de Docker:** La comunicación entre el servidor y los workers pasa por la red bridge virtual de Docker, que también tiene límites de throughput.

## Cómo identificar y medir los cuellos de botella

- docker stats — muestra CPU y memoria consumida por cada contenedor en tiempo real durante la prueba.
- htop — permite ver la carga general del sistema y el consumo del proceso Docker daemon.
- iostat -x 1 — muestra métricas de I/O de disco por dispositivo, para detectar saturación al levantar contenedores.